Cloud & Security Project Ideas

Oct 23, 2026 · @Bibek Tripathy

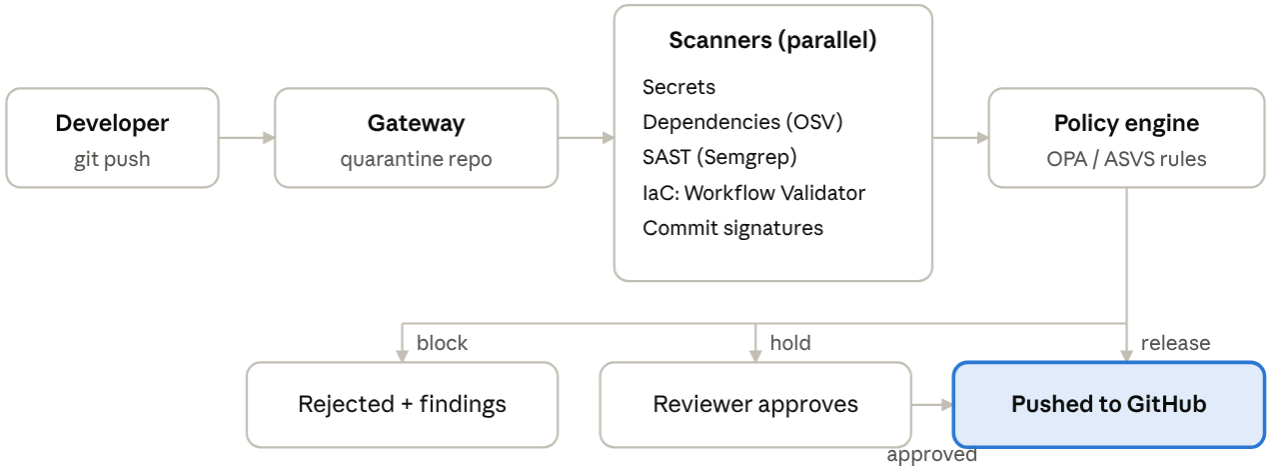
Flagship: Commit Quarantine Gateway (your DevSecOps idea)

Build it as a server-side git proxy, not just a hook: developers push to the gateway, it holds the push in quarantine, scans it, and only then forwards it to GitHub. Client-side pre-commit hooks can be skipped with `--no-verify`; a gateway in the push path cannot.

How a push flows

1. Developer runs `git push gateway main`. The gateway is a Go server speaking git's smart HTTP (or SSH) protocol.
2. It receives the packfile into a per-push quarantine bare repo. Nothing reaches the real remote yet.
3. It computes only the new commits (`old..new` range) and the changed files.
4. A worker pool of goroutines runs scanners in parallel, each a plugin behind one Go interface: secrets (entropy + regex rules), dependencies (OSV database), SAST (Semgrep), IaC/Dockerfile/Actions (your Cloud Workflow Validator), commit signature check.
5. Findings go to a policy engine (OPA/Rego or your ASVS rules) that returns **release**, **block** or **hold for approval**.
6. Release → gateway pushes the commits upstream with its own deploy key. Block → the push is rejected and the developer sees the findings in their terminal. Hold → a reviewer approves from a small dashboard.
7. Every verdict is written to an append-only audit log, with SARIF reports for each scan.

A push reaches GitHub only after the gateway releases it



commit quarantine gateway · push path

Scanners run side by side on the quarantined commits; only a release verdict forwards them upstream.

MVP (2–3 weeks): HTTP smart-git proxy, quarantine repo, secrets + dependency scanners, a YAML policy file, release/block, terminal output.

Stretch goals: hold-for-approval UI, Sigstore/gitsign signature verification, per-repo policies, Prometheus metrics (scan latency, block rate), a benchmark showing scan time vs. push size, and a red-team test suite of 50 planted secrets with detection rate reported.

Why it's strong for you: it reuses your Cloud Workflow Validator as a scanner plugin, shows Go concurrency and networking, and maps to real products (GitHub push protection, GitGuardian), so interviewers immediately get it.

Cloud infrastructure ideas

Project	What you build	Stack	Resume signal
Mini container orchestrator	Schedule containers across 3 home-lab nodes, health checks, restart on failure, bin-packing scheduler	Go, containerd API, gRPC, etcd or Raft	Distributed systems, scheduling

Project	What you build	Stack	Resume signal
Self-hosted PaaS ("mini Vercel")	`git push` → build image → deploy behind a reverse proxy with automatic TLS and preview URLs per branch	Go, Docker, Caddy/Traefik, webhooks	Platform engineering
Terraform drift detector	Compare live cloud state against Terraform state on a schedule, report and optionally auto-fix drift	Go or Python, AWS SDK, Terraform JSON plan	IaC, cloud ops
Cloud cost anomaly alerter	Pull daily billing data, flag spikes with a simple statistical model, alert to Slack/Discord	Python, AWS Cost Explorer, cron/Lambda	FinOps, data
GitOps home lab	Everything in your home lab declared in git and reconciled automatically, with dashboards	k3s, Argo CD or Flux, Prometheus, Grafana	Kubernetes, observability
Multi-region failover demo	Same app in two regions, health-checked DNS failover, measure recovery time	Terraform, AWS Route 53, load balancers	Reliability, SRE

Security tooling ideas

Project	What you build	Stack	Resume signal
Secrets scanner engine	Your own detector: regex + entropy + verified-key checks, benchmarked against gitleaks on a planted-secret corpus	Go	Detection engineering

Project	What you build	Stack	Resume signal
eBPF runtime monitor	Trace `execve`, file opens and outbound connects per container; alert on rules like "shell spawned in web container"	Go + cilium/ebpf, C for probes	Linux internals, runtime security
Kubernetes admission controller	Webhook that rejects pods running as root, with `:latest` images, or without resource limits	Go, controller-runtime	Policy-as-code, K8s
SBOM + vulnerability tracker	Generate SBOMs for every build, store them, alert when a new CVE hits a component you already shipped	Syft, OSV API, Postgres, Go	Supply-chain security
Zero-trust access proxy	Identity-aware reverse proxy in front of home-lab apps: OIDC login, per-app policy, mTLS to backends, audit log	Go, OIDC, mTLS	Zero-trust, auth
Honeypot + log pipeline	SSH/HTTP honeypot on a VPS, ship events to a dashboard, map attacker IPs and top credentials tried	Go or Python, Loki/Elastic, Grafana	Threat intel, logging
Cloud misconfig scanner	Read-only scan of an AWS account for public S3 buckets, open security groups, unused IAM keys, ranked by severity	Python/Go, AWS SDK	Cloud security posture

Systems and networking follow-ons

These extend work you've already done, so you can go deeper fast.

- **Lock-free data structures benchmark** (from the lock visualizer): add a lock-free queue and RCU-style reads, compare against mutex/spinlock on 1–32 threads, publish flame graphs.

- **SDN-based DDoS mitigation** (from the SDN load balancer): Ryu app that detects SYN floods from flow stats and installs drop rules automatically; measure time-to-mitigate in Mininet.
- **Userspace TCP stack** in C++ or Go on a TUN device: handshake, retransmission, Reno congestion control — pairs directly with your TCP congestion lab.
- **Distributed key-value store with Raft**: leader election, log replication, a chaos test that kills nodes; the classic backend interview project.
- **WireGuard mesh manager**: auto-configure a WireGuard mesh across your machines from one config file, with key rotation.

What to build first

Build the Commit Quarantine Gateway first, then one cloud and one security project that plug into it.

1. **Gateway MVP** — your flagship, and solo-owned, which matters since your recent projects are team repos.
2. **eBPF runtime monitor** or **K8s admission controller** — covers runtime security, the other half of DevSecOps.
3. **Self-hosted PaaS** or **GitOps home lab** — runs on hardware you already have, and can deploy the gateway itself.

For each: a README with an architecture diagram, one measured result (latency, detection rate, recovery time), and a short demo GIF. Those three things are what reviewers actually look at.